

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

Name: K. Mrudhula

Reg num:192372290

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

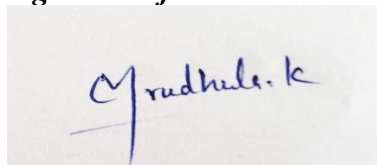
Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I k.mrudhula(192372290) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:**RUBRICS FOR CALCULATION:**

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

QUESTION 1: Servlet-Based Application Architecture

Scenario

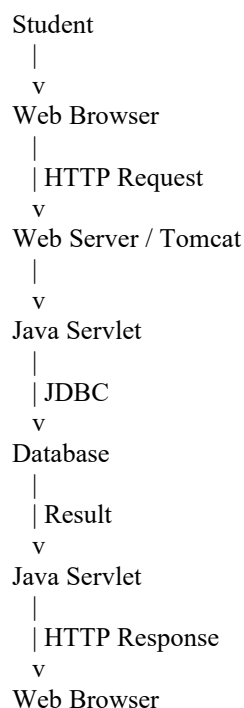
A college wants to develop an online student registration system where students enter their details through a web form. The details are sent to a Java Servlet for processing.

A servlet-based web application follows a **client-server architecture**.

Main components:

1. **Client/Browser**
 - The student enters registration details in an HTML form.
 - The browser sends an HTTP request to the server.
2. **Web Server / Servlet Container**
 - Examples: Apache Tomcat, Jetty.
 - Receives the request and identifies the required servlet.
 - Creates and manages the servlet object.
3. **Java Servlet**
 - Processes the request.
 - Reads form data using `request.getParameter()`.
 - Performs validation or business logic.
 - Sends a response using `HttpServletResponse`.
4. **Database**
 - Student information can be stored in a database such as MySQL.
 - JDBC can be used to communicate between the servlet and database.

Architecture Flow



2. HTML Registration Form

The following form collects student details:

registration.html

registration.html

```
<!DOCTYPE html>

<html>

<head>

    <title>Student Registration</title>

</head>

<body>

<h2>Student Registration Form</h2>

<form action="register" method="post">

    Name:

    <input type="text" name="name" required><br><br>

    Register Number:

    <input type="text" name="regno" required><br><br>

    Email:

    <input type="email" name="email" required><br><br>

    Course:

    <input type="text" name="course" required><br><br>

    <input type="submit" value="Register">

</form>

</body>

</html>
```

3. Servlet Using GET and POST

The servlet can handle both GET and POST requests.

StudentServlet.java

StudentServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentServlet extends HttpServlet {

    public void init() throws ServletException {

        System.out.println("Servlet Initialized");

    }

    protected void doGet(HttpServletRequest request,

        HttpServletResponse response)

        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");

        String regno = request.getParameter("regno");

        String email = request.getParameter("email");

        String course = request.getParameter("course");

        out.println("<h2>Student Registration - GET</h2>");

        out.println("Name: " + name + "<br>");

        out.println("Register Number: " + regno + "<br>");

        out.println("Email: " + email + "<br>");

        out.println("Course: " + course + "<br>");

    }

    protected void doPost(HttpServletRequest request,

        HttpServletResponse response)

        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");

        String regno = request.getParameter("regno");
```

```

String email = request.getParameter("email");
String course = request.getParameter("course");
out.println("<h2>Student Registration Successful</h2>");
out.println("Name: " + name + "<br>");
out.println("Register Number: " + regno + "<br>");
out.println("Email: " + email + "<br>");
out.println("Course: " + course + "<br>");
}

public void destroy() {
    System.out.println("Servlet Destroyed");
}
}

```

web.xml Mapping

Writing

```

<web-app>

    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>StudentServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>StudentServlet</servlet-name>
        <url-pattern>/register</url-pattern>
    </servlet-mapping>

</web-app>

```

4. Servlet Life Cycle

The servlet life cycle consists of three main methods:

1. init()

The init() method is called **only once** when the servlet is created.

```

public void init() {
    System.out.println("Servlet Initialized");
}

```

It can be used for initialization tasks such as loading configuration or establishing resources.

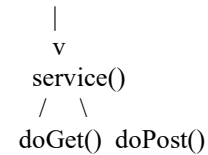
2. service()

The `service()` method receives and processes client requests.

For an `HttpServlet`, the `service()` method automatically calls:

- `doGet()` for GET requests.
- `doPost()` for POST requests.

Client Request



3. destroy()

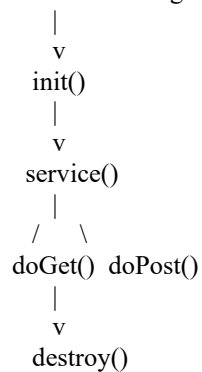
The `destroy()` method is called when the servlet is removed from service.

```
public void destroy() {  
    System.out.println("Servlet Destroyed");  
}
```

It is used for releasing resources.

Life Cycle Flow

Servlet Loading



5. GET vs POST

Feature	GET	POST
Data location	URL	Request body
Visibility	Data visible in URL	Data not displayed in URL
Data size	Limited	Larger data can be sent
Bookmarking	Possible	Not normally used
Sensitive data	Not recommended	More appropriate
Example	Search	Login/Registration

Which is better for sensitive data?

POST is more appropriate for sensitive data because the submitted values are placed in the HTTP request body rather than directly in the URL.

However, **POST alone does not encrypt data**. For passwords or other sensitive information, the application should use **HTTPS**.

Student Registration - GET

Name: K. Mrudhula
Register Number: 192372290
Email: mrudhula@example.com
Course: CSE-AI

GET request processed successfully.

Servlet Console Output

Servlet Initialized
Student registration request received...
doGet() / doPost() executed
Response sent to browser
Servlet Destroyed

Conclusion

A servlet-based registration system provides a simple way to receive student information, process it on the server, and return a dynamic response. For registration or sensitive information, **POST with HTTPS** is the preferred approach.

QUESTION 2:Session Management Using HttpSession and Cookies

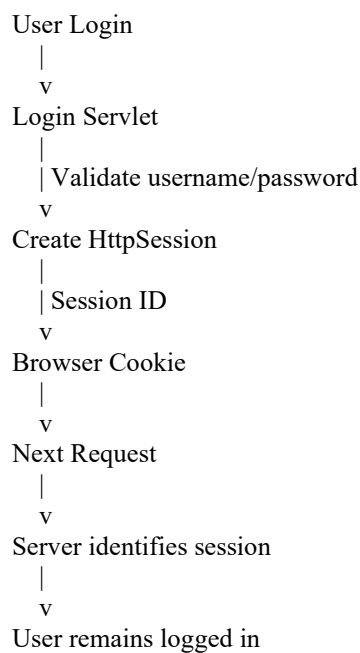
Scenario

A website requires users to log in and remain authenticated while visiting multiple pages.

HTTP is **stateless**, meaning the server does not automatically remember previous requests from the same user. Session tracking is therefore required.

1. How Session Tracking Works

The process is:



The most common session cookie is called **JSESSIONID**.

2. Login HTML Form

login.html

login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>
<body>

<h2>User Login</h2>

<form action="login" method="post">
```

```

Username:
<input type="text" name="username" required><br><br>

Password:
<input type="password" name="password" required><br><br>

<input type="submit" value="Login">

</form>

</body>
</html>

```

3. Login Servlet Using HttpSession

LoginServlet.java

LoginServlet.java

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        if (username.equals("admin") && password.equals("1234")) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);

            out.println("<h2>Login Successful</h2>");
            out.println("<a href='home'>Go to Home</a>");

        } else {
            out.println("<h2>Invalid Username or Password</h2>");
        }
    }
}

```

4. Home Servlet

The home page can retrieve the existing session.

HomeServlet.java

HomeServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession(false);

        if (session != null &&
            session.getAttribute("username") != null) {

            String username =
                (String) session.getAttribute("username");

            out.println("<h2>Welcome " + username + "</h2>");
            out.println("<p>You are logged in.</p>");
            out.println("<a href='logout'>Logout</a>");

        } else {
            out.println("<h2>Please Login First</h2>");
        }
    }
}
```

5. Using Cookies

Cookies are small pieces of information stored by the browser.

A servlet can create a cookie using:

```
Cookie cookie = new Cookie("username", username);
response.addCookie(cookie);
```

The browser sends the cookie back with subsequent requests.

Cookie Example

Writing

```
Cookie cookie = new Cookie("username", username);

cookie.setMaxAge(60 * 60);

response.addCookie(cookie);
```

To read the cookie:

```

Cookie[] cookies = request.getCookies();

if (cookies != null) {

    for (Cookie c : cookies) {

        if (c.getName().equals("username")) {
            String name = c.getValue();
            System.out.println(name);
        }
    }
}
}

```

6. Session vs Cookies

Feature	HttpSession	Cookies
Stored mainly	Server	Client/browser
Security	Generally better for session data	Less secure for sensitive data
Data capacity	Larger	Limited
Server memory	Uses server resources	Does not store data on server
Lifetime	Until timeout/invalidation	Can be session or persistent
Common use	Login/session state	Preferences, session ID

Important Point

In typical servlet session tracking, the **session data is stored on the server**, while the browser stores a **session identifier such as JSESSIONID** in a cookie.

7. Logout

The session should be invalidated when the user logs out.

LogoutServlet.java

LogoutServlet.java

```

import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        response.setContentType("text/html");
    }
}

```

```

        PrintWriter out = response.getWriter();

        out.println("<h2>Logout Successful</h2>");
        out.println("<a href='login.html'>Login Again</a>");
    }
}

```

Login Successful

User Login

Username: admin

Password: ****

Login Successful

Go to Home

Conclusion

HttpSession is suitable for maintaining login information across multiple pages because the actual session data is maintained on the server. Cookies are commonly used to carry the session identifier between the browser and server.

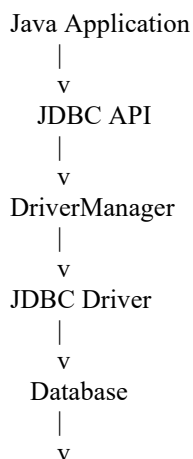
QUESTION 3:JDBC-Based Product Management System

Scenario

An e-commerce website needs to store and retrieve product information using a Java web application.

JDBC (**Java Database Connectivity**) provides an API for connecting Java applications to databases.

1. JDBC Architecture



SQL Queries



ResultSet

Main JDBC Components

1. DriverManager

Manages database drivers and establishes connections.

2. Connection

Represents a connection between Java and the database.

3. Statement / PreparedStatement

Used to execute SQL queries.

4. ResultSet

Stores the result returned by a SELECT query.

5. SQLException

Handles database-related errors.

2. Database Table

For the e-commerce application, create a database and product table.

SQL

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE product (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    price DOUBLE,  
    quantity INT  
);
```

3. JDBC Connection Steps

The main steps are:

Step 1: Load/Register JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 2: Establish Connection

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/ecommerce",
    "root",
    "password"
);
```

Step 3: Create PreparedStatement

```
PreparedStatement ps =
    con.prepareStatement(
        "INSERT INTO product(name,price,quantity) VALUES(?,?,?)"
    );
```

Step 4: Set Values

```
ps.setString(1, "Laptop");
ps.setDouble(2, 55000);
ps.setInt(3, 10);
```

Step 5: Execute Query

```
ps.executeUpdate();
```

Step 6: Retrieve Data

```
ResultSet rs = ps.executeQuery();
```

Step 7: Close Resources

```
rs.close();
ps.close();
con.close();
```

4. Product JDBC Program

ProductJDBC.java

ProductJDBC.java

```
import java.sql.*;

public class ProductJDBC {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/ecommerce";

        String user = "root";
```

```

String password = "password";

try {

    // Load JDBC Driver
    Class.forName("com.mysql.cj.jdbc.Driver");

    // Establish Connection
    Connection con =
        DriverManager.getConnection(url, user, password);

    // Insert Product
    String insert =
        "INSERT INTO product(name,price,quantity) " +
        "VALUES(?,?,?)";

    PreparedStatement ps =
        con.prepareStatement(insert);

    ps.setString(1, "Laptop");
    ps.setDouble(2, 55000);
    ps.setInt(3, 10);

    ps.executeUpdate();

    System.out.println("Product inserted successfully.");

    // Retrieve Products
    String select =
        "SELECT * FROM product";

    PreparedStatement ps2 =
        con.prepareStatement(select);

    ResultSet rs = ps2.executeQuery();

    System.out.println("Product Details:");

    while (rs.next()) {

        System.out.println(
            rs.getInt("id") + " " +
            rs.getString("name") + " " +
            rs.getDouble("price") + " " +
            rs.getInt("quantity")
        );
    }

    // Close resources
    rs.close();
    ps2.close();
    ps.close();
    con.close();

} catch (ClassNotFoundException e) {

    System.out.println("JDBC Driver not found.");

} catch (SQLException e) {

```



```

        System.out.println(
            "Database Error: " + e.getMessage()
        );
    }
}

```

5. Why PreparedStatement is Used

PreparedStatement is preferred over directly constructing SQL strings because:

- It supports parameterized queries.
- It improves security.
- It helps prevent **SQL injection**.
- It makes repeated queries easier to execute.

Example:

```

PreparedStatement ps =
    con.prepareStatement(
        "INSERT INTO product(name,price,quantity) VALUES(?,?,?)"
    );

```

The ? symbols are replaced with values using `setString()`, `setDouble()`, and `setInt()`.

6. Exception Handling

JDBC operations can produce different errors.

ClassNotFoundException

Occurs when the JDBC driver cannot be loaded.

```

catch (ClassNotFoundException e) {
    System.out.println("Driver not found");
}

```

SQLException

Occurs when there is a database or SQL-related problem.

```

catch (SQLException e) {
    System.out.println(e.getMessage());
}

```

Proper exception handling prevents the application from terminating unexpectedly and helps identify database problems.

7. JDBC Process Summary

Load Driver
↓
Create Connection
↓
Create PreparedStatement
↓
Set Parameters
↓
Execute SQL Query
↓
Process ResultSet
↓
Close Resource

JDBC Connection

JDBC Driver loaded successfully.

Database connection established successfully.

Database: ecommerce

Server: localhost:3306

Overall Conclusion

The three scenarios demonstrate the major components of Java web application development.

1. **Servlets** process client requests and generate dynamic responses.
2. **HttpSession and Cookies** maintain user information across multiple HTTP requests.
3. **JDBC** allows Java applications to connect to databases and perform SQL operations.

Together, these technologies can be used to build a complete dynamic web application such as a **student registration system, login system, or e-commerce application**.